PA3

CSE 3150 Fall 2025 - Song Han

Due 10/5/2025, 11:59pm.

Your goals in this assignment should be:
1. Get familiar with pointer and reference
2. Basic flow control (loop, etc.)
3. Implement basic algorithms

There are three components of this assignment. Each component asks you to write one C++ source file with the following names. Skeleton code has been provided.

1 ECRemoveMatch.cpp

The purpose of this assignment is to practice pointers and references. Here, you are given an array of pointers to strings. You need to implement a function called removeMatch that takes this array of pointers to strings and removes any matches with a duplicate pointer.
For example, suppose you are given an array of 3 pointers:

string s1 = "A", s2 = "A"
string *p1 = &s1, p2 = &s2, p3 = &s1
The array of pointers: array[] = {p1, p2, p3};

If you invoke: removeMatch(array, &s1), then the array should look like: array[] = {p2} since the strings pointed to are s1, s2, and s1 respectively.

Note: I will use C++ STL vector to represent an array. Essentially, STL vector is a "better" array. I want you to start to experiment with this powerful utility class. Google it to find references on how to use STL vector.

2 ECValidSudoku.cpp

Check if a given Sudoku board is a valid solution to the puzzle, returning true if it is or false if it is not. In this problem, we define a valid board state as one that has...

1. No duplicate values 1-9 in each **row**.
2. No duplicate values 1-9 in each **column**.
3. No duplicate values 1-9 in each **3x3 submatrix**.

The board does not need to be solvable to be valid; assess the board as it is in its current state, blank squares and all. This problem can be solved in O(n) time complexity.

Sudoku boards will be passed into the function as a 2D vector of string objects, with "." representing a blank square.

The following board state is **valid** because it contains no duplicate values 1-9 in any row, column, or 3x3 submatrix.

```
[["5","3",".",".","7",".",".",".","."]
,["6",".",".","1","9","5",".",".","."]
,[".","9","8",".",".",".",".","6","."]
,["8",".",".",".","6",".",".",".","3"]
,["4",".",".","8",".","3",".",".","1"]
,["7",".",".",".","2",".",".",".","6"]
,[".","6",".",".",".",".","2","8","."]
,[".",".",".","4","1","9",".",".","5"]
,[".",".",".",".","8",".",".","7","9"]]
```

The following board state is **invalid** because it contains two 8's in the top left 3x3 submatrix.

```
[["8","3",".",".","7",".",".",".","."]
,["6",".",".","1","9","5",".",".","."]
,[".","9","8",".",".",".",".","6","."]
,["8",".",".",".","6",".",".",".","3"]
,["4",".",".","8",".","3",".",".","1"]
,["7",".",".",".","2",".",".",".","6"]
,[".","6",".",".",".",".","2","8","."]
,[".",".",".","4","1","9",".",".","5"]
,[".",".",".",".","8",".",".","7","9"]]
```

Or more closely:

```
[["8","3","."
,["6",".","."
,[".","9","8",
```

3 ECNumOccurrences.cpp

You are given integer vectors A and B. You want to find the amount of times a number in A occurs in B. For this task, you are to implement a function called NumOccurrences, which takes two vectors and returns an vector C (whose indices map to A and contains the number of occurrences).

Note that B is sorted before being passed into the function.

For example, suppose A = {1, 3} and B = {1, 1, 2, 3, 6, 7, 10}.
Then, after NumOccurrences is run, C = {2, 1} indicating that "1" from A occurs twice and "3" occurs once.

As another example, suppose A = {1, 3, 1} and B = {1, 1, 1, 3, 3, 6, 7}.
Then, after NumOccurrences is run, C = {3, 2, 3} indicating that "1" from A occurs thrice, "3" occurs twice, and "1" occurs thrice again.

Things to remember when implementing this function:

1. You must implement the function as efficiently as possible. Recall that efficiency is one of the most important aspects of C++ programming. You should carefully think about the algorithm you are to use. The most straightforward algorithm is simply comparing each pair of numbers in the two vectors. However, this is very slow when the vectors get large. We will have run-time checks to ensure your code meets this criteria.

2. Don't reinvent the wheel! If you can, try to use the provided standard library functions (C++ STL). Search them up if you are not familiar with them.